Практическая работа №3. Форматы данных

0. ФИО, группа

ФИО: _____ Группа: _____

1. Ответы на вопросы

Чем отличаются текстовые и бинарные форматы?

Текстовые форматы (JSON, XML, YAML) хранят данные в виде читаемых символов вместе с именами полей, кавычками и скобками, поэтому их удобно отлаживать и править вручную, но они избыточны по размеру и медленнее разбираются. Бинарные форматы (MessagePack, CBOR, Protobuf) записывают данные в виде компактных байтовых кодов типов и длин: сообщение получается меньше и обрабатывается быстрее, но без специальных инструментов (и, для Protobuf, без файла схемы) его нельзя прочитать глазами.

Что такое схемная эволюция?

Схемная эволюция — это возможность менять структуру данных (добавлять и удалять поля) так, чтобы старые и новые версии программ продолжали понимать друг друга. В Protobuf поля идентифицируются номерами, поэтому новый код читает старые данные (отсутствующее поле получает значение по умолчанию), а старый код пропускает незнакомые поля новых данных.

2. Результаты выполнения шагов

Таблица 1. Сравнение текстовых форматов (размеры — по выводу step_json.py, step_xml.py, step_yaml.py)

Формат	Размер (байт)	Читаемость (1-5)	Есть ли комментарии?
JSON	253	4	Нет
XML	283	3	Да (<!-- -->)
YAML	182	5	Да (#)

Примечание: в step_json.py используется форматирование indent=2 (253 байта); без отступов, как в run_all.py, тот же JSON занимает 211 байт.

Таблица 2. Сводная таблица всех форматов (по выводу run_all.py)

Формат	Размер (байт)	Читаем ли человеком?	Нужна ли схема?
JSON	211	Да	Нет
XML	283	Да	Нет
YAML	182	Да	Нет
MessagePack	157	Нет	Нет
CBOR	159	Нет	Нет
Protobuf	117	Нет	Да

Скриншоты

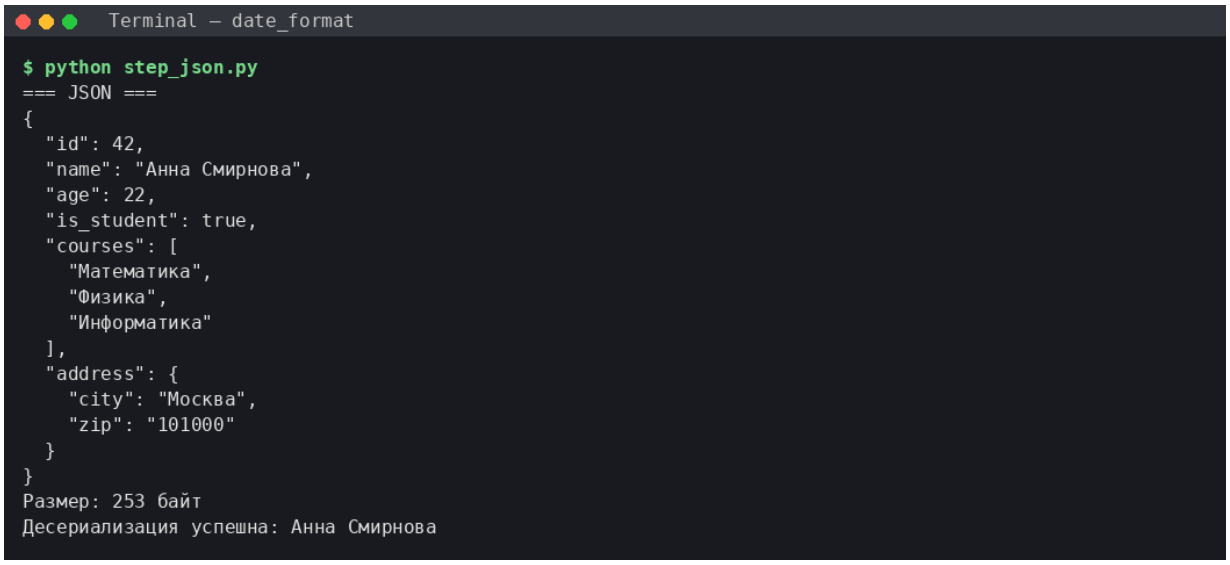


Рисунок 1 — Вывод консоли step_json.py

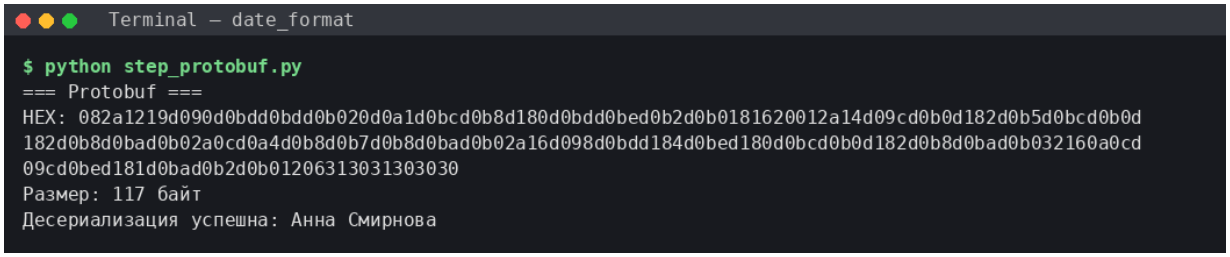


Рисунок 2 — Вывод консоли step_protobuf.py

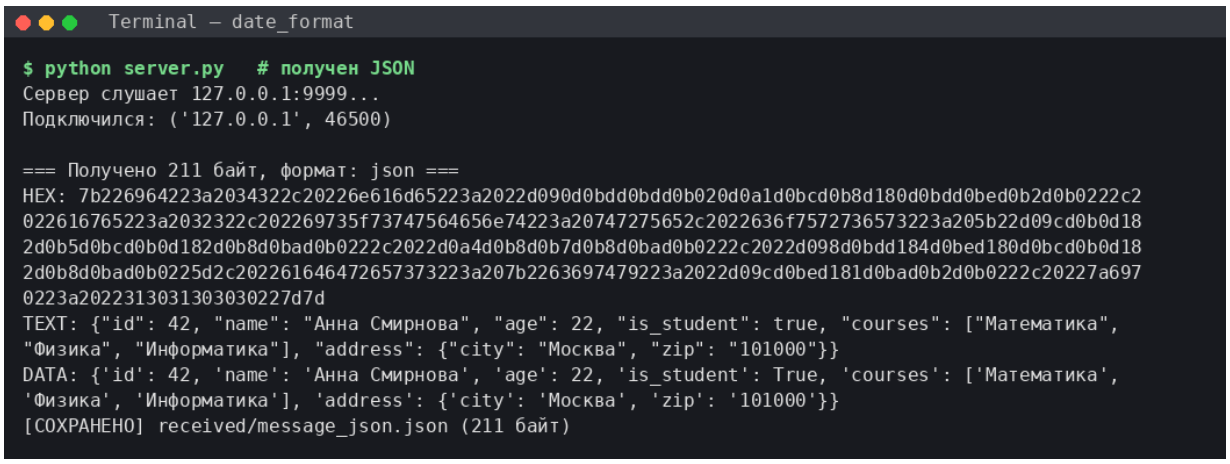


Рисунок 3 — Сервер: блок «=== Получено ... ===» для JSON (TEXT читаемый)

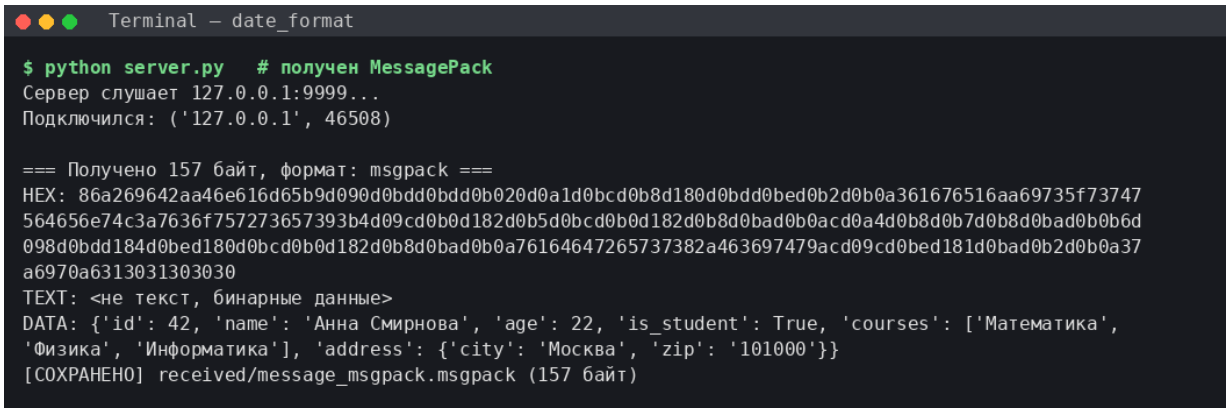


Рисунок 4 — Сервер: блок «=== Получено ... ===» для MessagePack (TEXT: <не текст, бинарные данные>)

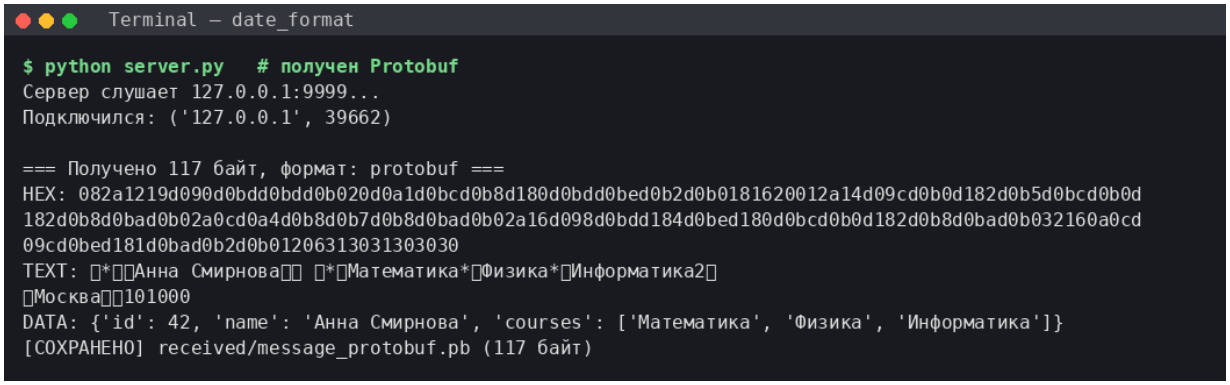


Рисунок 5 — Сервер: блок «=== Получено ... ===» для Protobuf (TEXT — нечитаемый набор символов)

Примечание к рисунку 5: байты Protobuf-сообщения здесь случайно оказались корректной UTF-8 последовательностью (в основном из-за кириллицы), поэтому server.py не вывел «<не текст, бинарные данные>», а напечатал нечитаемую смесь управляющих символов и текста. Понятного человеку содержимого в строке TEXT всё равно нет, а в MessagePack и CBOR сработала ветка «<не текст>».

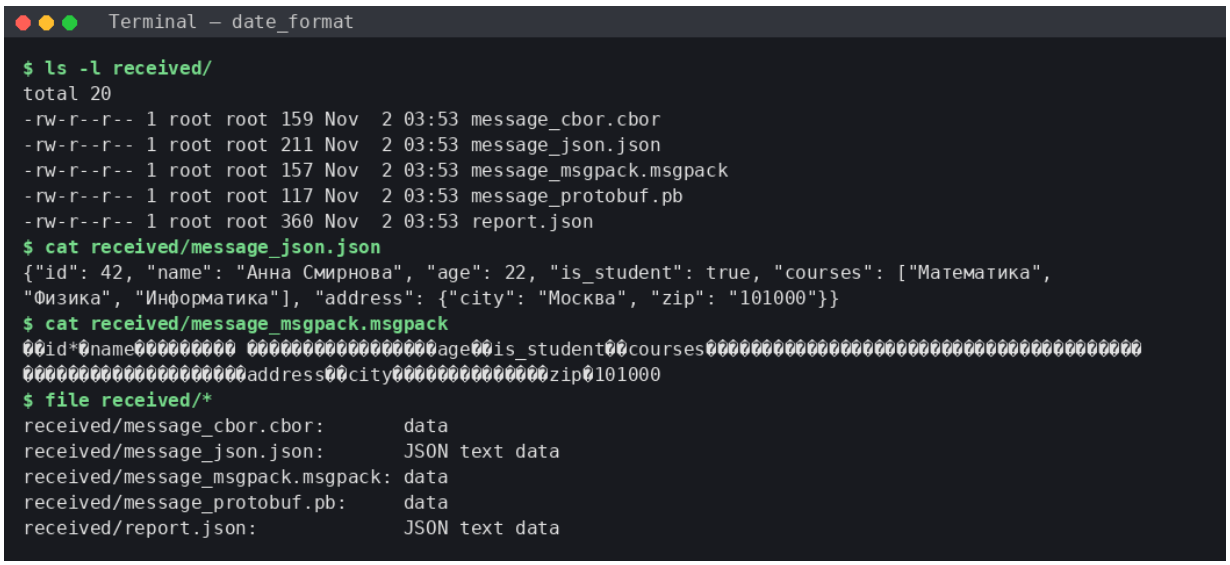


Рисунок 6 — Папка received/: JSON открывается как текст, .msgpack и .pb — нет

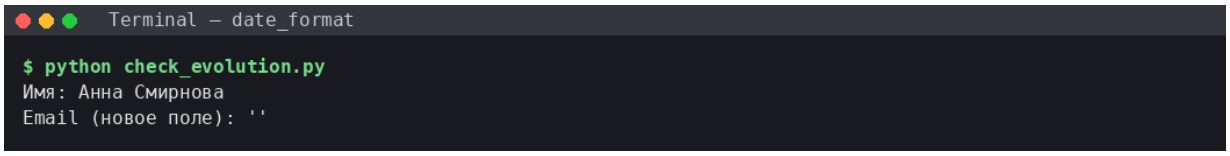


Рисунок 7 — Проверка схемной эволюции: старые данные прочитаны, email — пустая строка

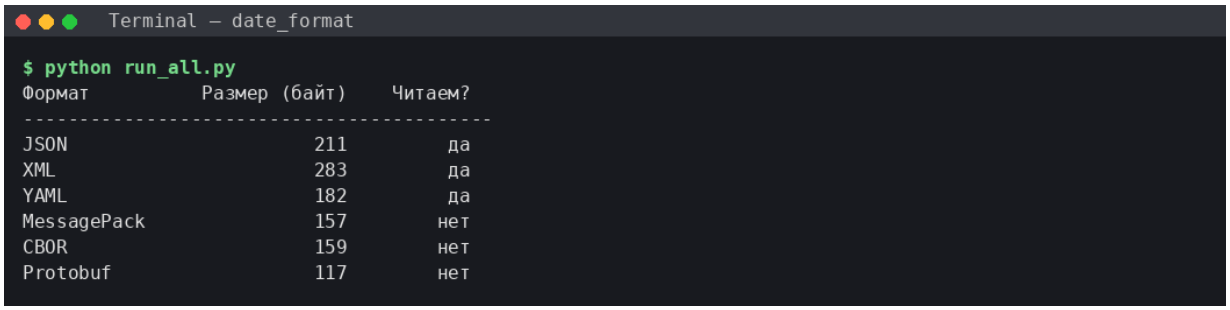


Рисунок 8 — Вывод сводного отчёта run_all.py

3. Выводы

1. Почему Protobuf и MessagePack оказались меньше JSON?

В JSON каждое сообщение содержит имена полей в кавычках, двоеточия, запятые, скобки и пробелы, а числа и булевы значения записаны текстом (true, 42). В MessagePack типы и длины кодируются одним-двумя байтами, а числа хранятся в двоичном виде, поэтому лишняя «обвязка» исчезает (211 → 157 байт), хотя имена полей по-прежнему записываются строками. В Protobuf имён полей в сообщении нет вовсе — вместо них используются номера полей из схемы (.proto), которая хранится отдельно, поэтому результат самый компактный (117 байт). Заметная часть размера у всех форматов — это сам текст на кириллице (около 97 байт в UTF-8), который сжать форматом нельзя.

2. В каком формате данные было легче всего читать вручную?

Легче всего читать YAML и JSON: в YAML нет кавычек и скобок, структура видна по отступам, а JSON с отступами тоже понятен и привычен. XML

читается тяжелее из-за повторяющихся открывающих и закрывающих тегов, а бинарные форматы (MessagePack, CBOR, Protobuf) в HEX-виде вручную практически не читаются.

3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?

У JSON нет формальной схемы: структуру определяют только имена ключей внутри самих данных, поэтому нет договора о том, какие поля обязательны, какого они типа и что делать с отсутствующим полем. Если код ожидает новый ключ, а в старых данных его нет, он получит ошибку (KeyError) или None, и каждую такую ситуацию программист обязан обрабатывать вручную; переименование поля ломает совместимость. В Protobuf поля однозначно определяются номерами в схеме: новое поле с новым номером (email = 7) можно добавить, и новый код читает старые данные без ошибок (отсутствующее поле получает значение по умолчанию — пустую строку), а старый код просто пропускает неизвестные ему поля. Условие — не менять и не переиспользовать уже занятые номера полей.

Чек-лист

- Все .py файлы и .proto файл присутствуют.
- Сгенерирован profile_pb2.py.
- Сделаны скриншоты вывода консоли.
- Заполнены все таблицы.
- Сделан и обоснован вывод.